SEGUNDO DEPARTAMENTAL – Ejercicio 19 – Ordenamiento burbuja indirecto (PE)

Introducción

Este programa ordena indirectamente un conjunto de números. En lugar de mover los datos originales, ordena un arreglo auxiliar de índices que indica el orden en que deben consultarse los valores.

Código completo

```
#include <iostream>
using namespace std;

const int MAX = 100;

// Funcion para ingresar los numeros
void ingresarDatos(int datos[], int n) {
    cout << "Ingresa " << n << " numeros enteros:" << endl;

    for (int i = 0; i < n; i++) {
        cin >> datos[i];
    }
}

// Funcion para inicializar los indices
void inicializarIndices(int indices[], int n) {
    for (int i = 0; i < n; i++) {
        indices[i] = i;
    }
}

// Ordenamiento burbuja indirecto
void ordenarIndirecto(int datos[], int indices[], int n) {
    int auxiliar;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {

            if (datos[indices[j]] > datos[indices[j + 1]]) {
                auxiliar = indices[j];
                indices[j] = indices[j + 1];
                indices[j + 1] = auxiliar;
            }
        }
    }
}

// Funcion para mostrar los datos originales
void mostrarOriginales(int datos[], int n) {
    for (int i = 0; i < n; i++) {
        cout << datos[i] << " ";
    }
    cout << endl;
}

// Mostrar los datos utilizando los indices ordenados
void mostrarOrdenados(int datos[], int indices[], int n) {
    for (int i = 0; i < n; i++) {
        cout << datos[indices[i]] << " ";
    }
    cout << endl;
}

int main() {
```

```
    int datos[MAX];
    int indices[MAX];
    int n;

    cout << "ORDENAMIENTO INDIRECTO POR BURBUJA" << endl;
    cout << "Cuantos numeros deseas ordenar? ";
    cin >> n;

    if (n < 1 || n > MAX) {
        cout << "Cantidad no valida." << endl;
        return 1;
    }

    ingresarDatos(datos, n);
    inicializarIndices(indices, n);

    cout << "\nDatos originales: ";
    mostrarOriginales(datos, n);

    ordenarIndirecto(datos, indices, n);

    cout << "Datos ordenados indirectamente: ";
    mostrarOrdenados(datos, indices, n);

    cout << "\nIndices ordenados: ";
    for (int i = 0; i < n; i++) {
        cout << indices[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Explicación del código

ingresarDatos() captura los números; inicializarIndices() llena el arreglo de índices con las posiciones originales. ordenarIndirecto() compara los datos a través de esos índices e intercambia únicamente los índices. Las funciones de visualización muestran los datos originales, los ordenados indirectamente y los índices.

Funcionamiento

El usuario introduce entre uno y cien números. El programa conserva el arreglo original y presenta el orden lógico obtenido mediante los índices.

Elementos y conceptos principales

Ordenamiento burbuja, arreglos paralelos, índices y acceso indirecto a los elementos.

Conclusión del programa

Con este programa se practica la aplicación de los conceptos indicados y se comprende cómo se organiza una solución en C++. Su ejecución permite observar la relación entre las instrucciones, los datos de entrada y los resultados que se muestran en pantalla.

SEGUNDO DEPARTAMENTAL – Ejercicio 19 – Ordenamiento burbuja indirecto (POO)

Introducción

Este programa aplica el ordenamiento burbuja indirecto utilizando una clase. Los valores originales se conservan y el orden se representa mediante un arreglo de índices.

Código completo

```
#include <iostream>
using namespace std;

class OrdenamientoIndirecto {
private:
    int datos[100];
    int indices[100];
    int cantidad;

public:
    // Constructor
    OrdenamientoIndirecto(int n) {
        cantidad = n;
    }

    // Ingresar datos
    void ingresarDatos() {
        cout << "Ingresa " << cantidad
             << " numeros enteros:" << endl;

        for (int i = 0; i < cantidad; i++) {
            cin >> datos[i];
            indices[i] = i;
        }
    }

    // Ordenamiento burbuja indirecto
    void ordenarBurbuja() {
        int auxiliar;

        for (int i = 0; i < cantidad - 1; i++) {
            for (int j = 0; j < cantidad - 1 - i; j++) {

                if (datos[indices[j]] >
                    datos[indices[j + 1]]) {

                    auxiliar = indices[j];
                    indices[j] = indices[j + 1];
                    indices[j + 1] = auxiliar;
                }
            }
        }
    }

    // Mostrar datos originales
    void mostrarOriginales() {
        for (int i = 0; i < cantidad; i++) {
            cout << datos[i] << " ";
        }
        cout << endl;
    }

    // Mostrar datos ordenados indirectamente
    void mostrarOrdenados() {
        for (int i = 0; i < cantidad; i++) {
```

```
            cout << datos[indices[i]] << " ";
        }
        cout << endl;
    }

    // Mostrar indices ordenados
    void mostrarIndices() {
        for (int i = 0; i < cantidad; i++) {
            cout << indices[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    int n;

    cout << "ORDENAMIENTO INDIRECTO CON POO" << endl;
    cout << "Cuantos numeros deseas ordenar? ";
    cin >> n;

    if (n < 1 || n > 100) {
        cout << "Cantidad no valida." << endl;
        return 1;
    }

    // Crear objeto
    OrdenamientoIndirecto objeto(n);

    objeto.ingresarDatos();

    cout << "\nDatos originales: ";
    objeto.mostrarOriginales();

    objeto.ordenarBurbuja();

    cout << "Datos ordenados indirectamente: ";
    objeto.mostrarOrdenados();

    cout << "Indices ordenados: ";
    objeto.mostrarIndices();

    return 0;
}
```

Explicación del código

La clase OrdenamientoIndirecto contiene los arreglos datos e indices y la cantidad. ingresarDatos() captura los números e inicializa los índices; ordenarBurbuja() intercambia índices; los métodos de mostrar imprimen datos originales, ordenados e índices. En main() se valida la cantidad y se utiliza un objeto.

Funcionamiento

Se captura la cantidad y los números, se muestran los datos originales, se realiza el ordenamiento indirecto y se imprimen los datos ordenados y sus índices.

Elementos y conceptos principales

POO, constructor, arreglos, índices, métodos y ordenamiento indirecto.

Conclusión del programa

Con este programa se practica la aplicación de los conceptos indicados y se comprende cómo se organiza una solución en C++. Su ejecución permite observar la relación entre las instrucciones, los datos de entrada y los resultados que se muestran en pantalla.